AIORI
Hackathon 3

Advanced Internet Operations Research in India

A3-PS023-TC213

Minimal Two-Node Cloud Hypervisor VM Controller

Problem Statement PS-023 · Cloud Computing

Team Episteme · MAKAUT, WB

FACULTY MENTOR / LEAD ADVISOR
Sayani Manna Assistant Professor, Department of Information Technology, MAKAUT, WB

STUDENT TEAM LEAD · Controller & Placement Architect
Soham Chakraborti soham.py.code@gmail.com B.Tech. IT, 2nd Year, MAKAUT, WB

TEAM MEMBER · Worker & Hypervisor Integration
Ishita Roy royishita2006@gmail.com B.Tech. IT, 2nd Year, MAKAUT, WB

AIORI Hackathon 3

October 14, 2026

Problem Description

Why a lightweight two-node VM controller is needed

- **Over-weight orchestration:** OpenStack or Kubernetes + KubeVirt need a database, a message bus and many daemons - far too heavy to manage two KVM hosts and a handful of VMs.
- **Orphan-process risk:** every VM is a standalone cloud-hypervisor process. If the agent crashes, the VM process, its API socket, TAP device and disk copy keep leaking RAM, IPs and disk.
- **Racing placement:** two simultaneous creates can read the same "free memory" and over-commit one host.
- **Unenforced limits:** a misbehaving guest can exceed its share unless limits are enforced by the host (cgroup v2), not only by API validation.
- **Partial failures:** a failed create or start must roll back completely; a dead worker must never receive new VMs.
- **Weak control plane:** an unauthenticated or injectable API lets any caller start processes on the host.

Our Objective

- A tiny **mini-cloud**: one Controller + two Worker Agents (one per KVM host)
- Authenticated REST API: create, start, stop, delete, list
- Memory-based placement with host reserve
- Host-enforced CPU and memory limits
- **Zero** orphaned processes, sockets, TAPs or disks

Explicit non-goals: live migration, distributed storage, browser VNC, image marketplace, a new hypervisor.

Background

KVM, Cloud Hypervisor and the Controller-Worker model

1. KVM and /dev/kvm

- Linux kernel module that exposes CPU hardware virtualization (Intel VT-x / AMD-V) through the **/dev/kvm** device.
- A VMM opens /dev/kvm and uses ioctls to create the VM, vCPUs and memory slots; guest code then runs natively on the CPU.
- Day-1 check: /dev/kvm must exist and be accessible (nested virtualization on cloud instances).

2. Cloud Hypervisor (CH)

- Lightweight Rust VMM built on KVM with a minimal device model and low idle footprint.
- **One VM = one cloud-hypervisor OS process**, started with --api-socket path=...
- Config: boot/max vCPUs, fixed memory size, kernel or firmware, disk, TAP network.

3. REST over a Unix domain socket

- Each CH process serves an HTTP REST API on a Unix socket: no TCP port is exposed; access is guarded by filesystem permissions.
- Endpoints: vm.create, vm.boot, vm.shutdown, vm.delete, vm.info, vmm.ping, vmm.shutdown.
- Worker calls them with httpx using a Unix-socket transport.

4. Controller vs. Worker roles

- **Controller (brain):** worker registry, placement, VM records in SQLite, public authenticated API.
- **Worker (hands):** one per host; spawns CH, talks to sockets, reports free memory, keeps a local registry (pid, socket, TAP, disk).
- Two trust boundaries: Bearer API key for clients; shared secret or mTLS between controller and workers.

Literature & RFC Review

Standards that shape our control plane, security and isolation

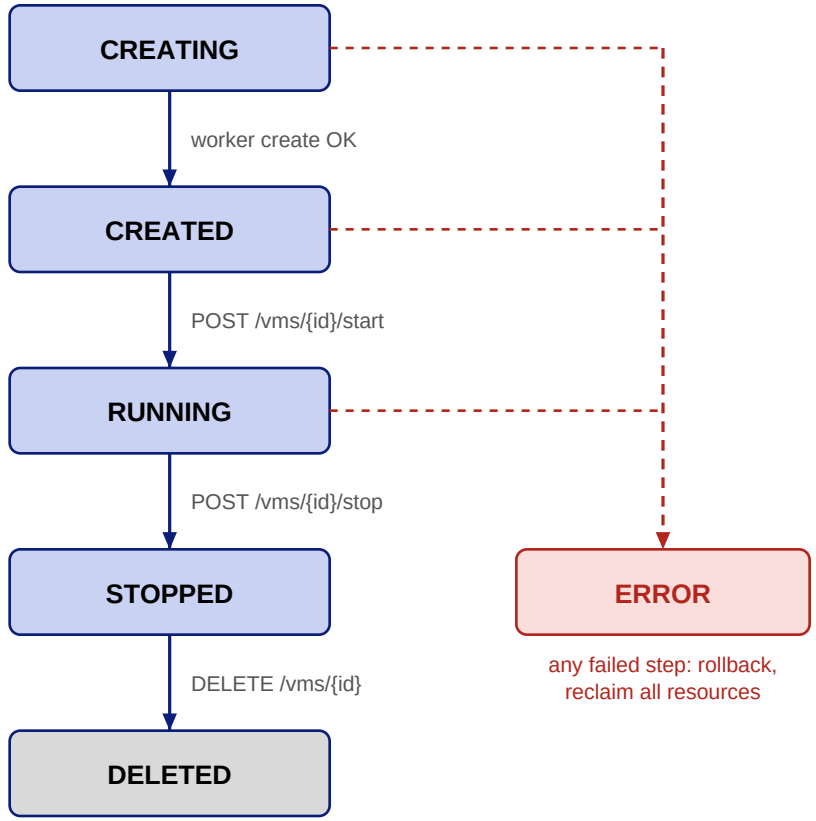
Standard / Source	What it specifies	How we apply it
RFC 7235 [1] HTTP/1.1 Authentication (now folded into RFC 9110 [2])	401 Unauthorized with a WWW-Authenticate challenge; the Authorization request header framework.	Missing or wrong credentials return 401; covered by the wrong-token failure-injection test.
RFC 6750 [3] OAuth 2.0 Bearer Token Usage	Syntax "Authorization: Bearer token" and guidance to keep tokens out of URLs and logs.	Client API key on every controller endpoint; secrets from env or files, never committed; constant-time compare.
RFC 8446 [4] TLS 1.3	Authenticated key exchange, AEAD encryption, optional client-certificate authentication (mutual TLS).	Encrypted controller-to-worker channel; mTLS as hardening; workers bound to the private interface only.
RFC 9110 [2] HTTP Semantics, **RFC 4918** [5]	409 Conflict; method idempotency (DELETE); 507 Insufficient Storage.	Illegal state transition returns 409; stop on a STOPPED VM returns 200; no-capacity placement returns 507.
Linux cgroups v2 [6] kernel admin guide	Unified hierarchy with memory.max and cpu.max (quota/period); a group can be killed as a unit.	Each CH runs in its own scope (systemd-run --scope -p MemoryMax= -p CPUQuota=): limits hold even if the guest misbehaves.
POSIX.1-2017 [7] process groups and signals	setsid()/setpgid(), kill(-pgid, sig), catchable SIGTERM vs uncatchable SIGKILL, waitpid().	CH has its own process group; teardown escalates vm.shutdown, vm.delete, vmm.shutdown, SIGTERM, then SIGKILL after a timeout.
KVM API [8][9], **Cloud Hypervisor API** [10][11]	/dev/kvm ioctl interface; documented vm.* and vmm.* REST endpoints.	Only documented calls are used; no hypervisor changes (a new hypervisor is a non-goal).

Proposed Solution

Controller + two Worker Agents with a deterministic VM state machine

- **Topology:** one Controller and two Worker Agents (one per KVM host). Users call an HTTPS REST API with a Bearer key.
- **Stack:** Python 3.11, FastAPI (controller and worker APIs), httpx (worker calls and Unix-socket transport to CH), SQLite (VM records), pytest. No shell=True anywhere.
- **Controller:** auth, validation (1-4 vCPU, 128-2048 MB), placement, VM records, heartbeats, retries with backoff.
- **Worker:** per-VM disk copy, TAP/IP allocation, spawn CH in a cgroup v2 scope and its own process group, CH API calls, reconciler.
- **Placement:** healthy host with most free memory that fits request + 512 MB reserve; tie-break by fewer VMs; if none fits, reject with 507.
- **Deterministic lifecycle:** only legal transitions are allowed; anything else returns HTTP 409; operations are idempotent.

VM lifecycle state machine



Idempotent: stop on an already STOPPED VM returns OK.
Invalid transitions (e.g. start on DELETED) return 409.

Optimization Proposed

Naive controller vs. our engineered solution

Aspect	Naive controller	Our engineered solution	Impact
Process isolation	CH spawned as a plain child; shares the agent's fate; no host limits.	Own process group + cgroup v2 scope (MemoryMax, CPUQuota). Stop = vm.shutdown, vm.delete, vmm.shutdown, SIGTERM, SIGKILL.	Guest cannot exceed limits; whole VM tree is killable.
Memory reservation locks	Reads free memory, then creates: concurrent requests over-commit a host.	Serialized placement lock + reserved-in-flight accounting + 512 MB host reserve.	No placement race; clean 507 when full.
Failure rollback	A failed create leaves socket, TAP, disk copy and registry entry behind.	Every step has an undo: kill process, remove socket, delete TAP, remove disk copy, drop registry entry.	No leaks after any failed step.
Cleanup reconciler	Orphans are found and removed by hand.	Runs at worker start and periodically: kills stray CH PIDs, sockets and TAPs not in the registry; fixes dead-but-RUNNING VMs.	Self-healing; leak_check.sh reports zero.
Failover	Calls hang; a dead host can still be chosen.	Timeouts + retries with backoff; host UNHEALTHY after N missed heartbeats; never place on it; reconcile VM state on return.	Controller stays responsive and safe.
Safety and security	Repeated stop raises errors; one static token; shell commands built from input.	Idempotent operations; separate client key and worker secret; constant-time compare; strict input validation, no shell=True.	Safe retries; no path or command injection.

Solution Architecture & Design

Component diagram: User/CLI -> Controller -> Worker A/B -> Cloud Hypervisor

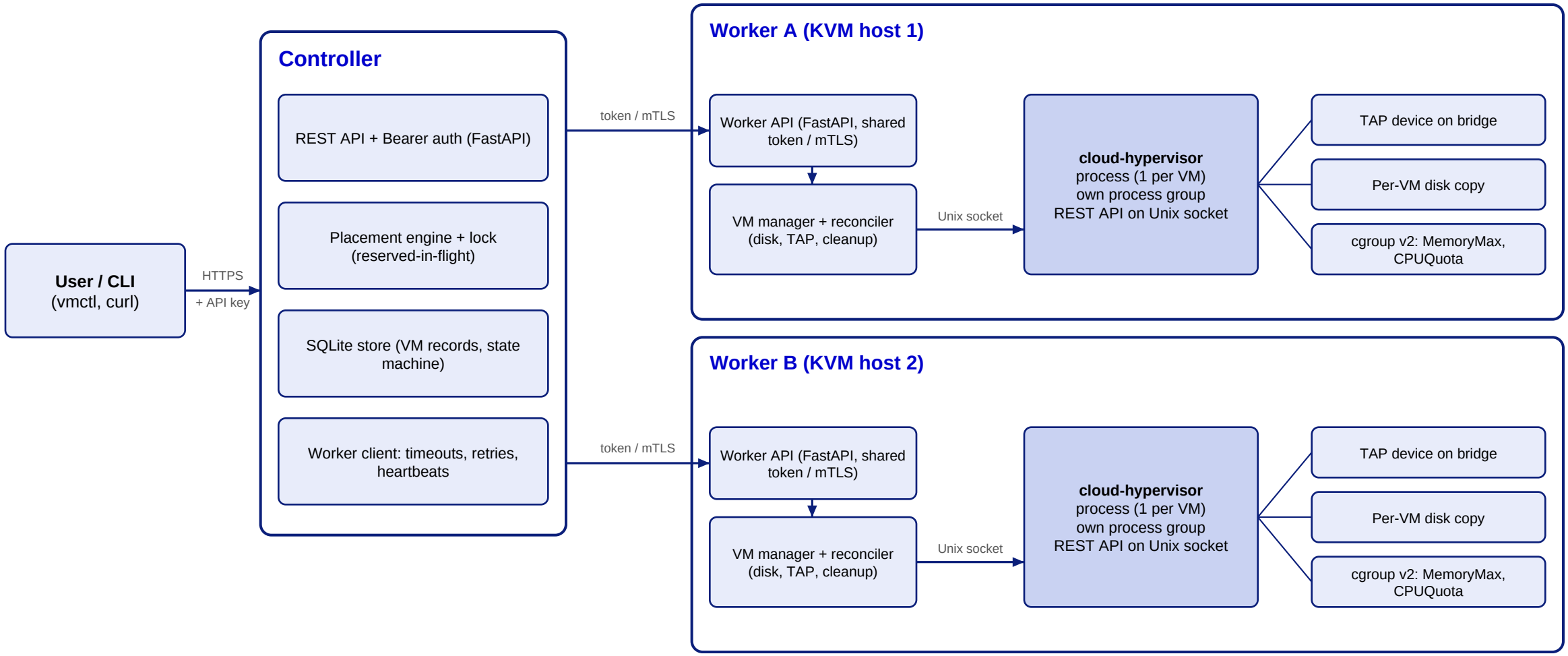
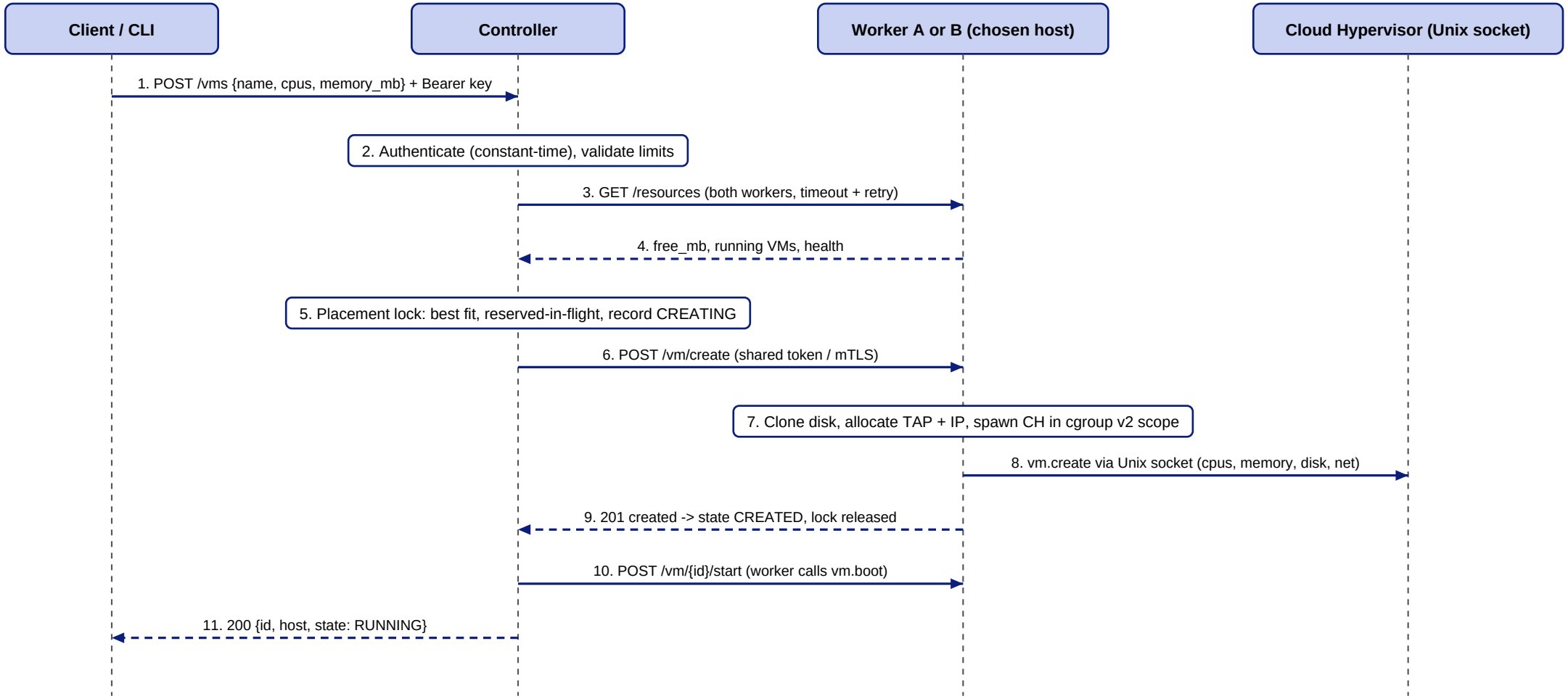


Figure: component diagram of the PS-023 mini-cloud (one cloud-hypervisor process per VM).

Data Flow Diagram

Sequence of a VM create-and-start request (POST /vms)



Failure at steps 6-8: rollback (kill process group, remove socket, TAP and disk copy, drop registry entry), mark ERROR, release reserved memory.

Deployment Timeline

Milestones M0-M7 and delivery methodology

Milestone	Scope	Exit criterion	Owner
M0 Setup	Repo, access to 2 KVM hosts, confirm /dev/kvm, install Cloud Hypervisor, boot one VM by hand from the CLI.	One manually booted VM; environment notes.	All
M1 Worker core	Launch, boot and stop one VM through the CH API socket from Python (ch_client.py).	Scripted create, boot, shutdown on one host.	Ishita Roy
M2 Worker API	Worker REST API, shared-secret auth, rollback and process-group teardown.	Lifecycle test passes with zero leftovers.	Ishita Roy
M3 Controller	FastAPI endpoints, SQLite VM records, Bearer auth, state machine.	Create/start/stop/list/delete via controller.	Soham Chakraborti
M4 Placement + limits	/resources polling, memory placement with lock and reserve, validation, cgroup v2 limits.	Host A full -> VM lands on B; both full -> rejection.	Soham + Ishita
M5 Failure handling	Reconciler, heartbeats, UNHEALTHY marking, retries with backoff, idempotency.	kill -9 of CH and worker-down scenarios self-heal.	Soham + Ishita
M6 Tests + leak checker	pytest unit, lifecycle and failure-injection suites; leak_check.sh (pgrep, /run sockets, ip link).	make test green; zero leaks.	All
M7 Report + demo	PDF report, 1-page pseudocode, 5-7 min demo rehearsal, repo handover.	Submission package ready.	All; review by Sayani Manna

Methodology: learn-by-doing (M0 manual first), feature branches feature/controller and feature/agent with peer review before merge to main, early integration tests for API failures and memory constraints, regular syncs on the controller-worker API contract.

Bibliography

References

[1] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Authentication," RFC 7235, IETF, Jun. 2014 (obsoleted by RFC 9110).

[2] R. Fielding, M. Nottingham and J. Reschke (Eds.), "HTTP Semantics," RFC 9110, IETF, Jun. 2022.

[3] M. Jones and D. Hardt, "The OAuth 2.0 Authorization Framework: Bearer Token Usage," RFC 6750, IETF, Oct. 2012.

[4] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, Aug. 2018.

[5] L. Dusseault (Ed.), "HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)," RFC 4918, IETF, Jun. 2007 (507 Insufficient Storage).

[6] Linux Kernel Documentation, "Control Group v2," Documentation/admin-guide/cgroup-v2.rst. https://docs.kernel.org/admin-guide/cgroup-v2.html

[7] IEEE and The Open Group, "IEEE Std 1003.1-2017 (POSIX.1-2017), System Interfaces: setsid, setpgid, kill, waitpid," 2018.

[8] Linux Kernel Documentation, "The Definitive KVM (Kernel-based Virtual Machine) API Documentation," Documentation/virt/kvm/api.rst.

[9] A. Kivity, Y. Kamay, D. Laor, U. Lublin and A. Liguori, "kvm: the Linux Virtual Machine Monitor," in Proc. Linux Symposium, vol. 1, 2007, pp. 225-230.

[10] Cloud Hypervisor Project (Linux Foundation), "Cloud Hypervisor: A Virtual Machine Monitor for Modern Cloud Workloads," source code and documentation. https://github.com/cloud-hypervisor/cloud-hypervisor

[11] Cloud Hypervisor Project, "Cloud Hypervisor REST API (vm.create, vm.boot, vm.shutdown, vm.delete, vm.info, vmm.ping, vmm.shutdown)," docs/api.md in the project repository.

Thank You for Your Attention!

Team Episteme

MAKAUT, WB

Sayani Manna (Faculty Mentor) · Soham Chakraborti (Student Team Lead) · Ishita Roy

A3-PS023-TC213 · PS-023 Minimal Two-Node Cloud Hypervisor VM Controller · AIORI Hackathon 3